CTMSCS/DFIS SIII P4/P5 EL3 Database Security/ Forensics **Odd 2026**

Lecture Note#1: SQL-DDL, DML and Built-in Functions

Lecturer: Dr. Himanshu B Patel *Scribes: M.Sc CS/DFIS 2nd year, Semester III*

Note 1: *LaTeX template courtesy of UC Berkeley EECS dept.*
Note 2: *This documet might contain hyperlinks that require active internet connection.*
Disclaimer: *This lecture note is prepared for the students of the School of Cybersecuirity and Digital Forensics, National Forensic Sciences University and has not been subjected to the usual scrutiny reserved for formal publications. They may be distributed outside this class only with the permission of the Instructor.*

Relational databases are not only theory; they live through **SQL statements** that:

- Define the **structure** of data (**DDL** – Data Definition Language),
- Manipulate the **contents** (**DML** – Data Manipulation Language),
- Retrieve and transform data for humans and applications (**SELECT** and **functions**),
- Control transactions and consistency (**COMMIT**, **ROLLBACK**).

By the end of this unit, you should be able to:

- Create, modify and drop tables with appropriate constraints in Oracle.
- Insert, update, delete and query data using LiveSQL.
- Use single-row functions (character, number, date, conversion, general functions).
- Understand the flow between definition, manipulation and retrieval of data.

1.1 Our Common Practice Schema

To get continuous hands-on experience, we will use the same small schema in **all remaining DBMS units**.

1.1.1 Tables Used in All Units

We will use four core tables:

- `DEPARTMENT`: Master data for departments.
- `EMPLOYEE`: Employee details (with salary & manager relationship).
- `PROJECT`: Projects handled by departments.
- `EMP_PROJECT`: Many–to–many relationship between employees and projects.

[0]These notes are partially based on the book "Introduction to Languages and the Theory of Computation" by John C Martin (Fourth Edition)

1.1.2 Creating the Schema in Oracle LiveSQL

Instructions to students:

1. Open https://livesql.oracle.com and log in.
2. Open a new **SQL Worksheet**.
3. Copy–paste the following script and execute it **ONCE**.
   (If you already created tables with the same names in previous labs, either use a fresh workspace or drop the old tables.)

```
-- STEP 1: Drop tables if they exist (may give error on first run; ignore it)
DROP TABLE EMP_PROJECT;
DROP TABLE PROJECT;
DROP TABLE EMPLOYEE;
DROP TABLE DEPARTMENT;
```

Note: In LiveSQL, DROP will give an error if the table does not exist. That is fine for our practice.

```
-- STEP 2: Create master table DEPARTMENT
CREATE TABLE DEPARTMENT (
DEPT_ID NUMBER PRIMARY KEY,
DEPT_NAME VARCHAR2(50) NOT NULL,
LOCATION VARCHAR2(30)
);

-- STEP 3: Create EMPLOYEE table
CREATE TABLE EMPLOYEE (
EMP_ID NUMBER PRIMARY KEY,
FIRST_NAME VARCHAR2(30) NOT NULL,
LAST_NAME VARCHAR2(30) NOT NULL,
EMAIL VARCHAR2(50) UNIQUE,
PHONE_NO VARCHAR2(15),
HIRE_DATE DATE DEFAULT SYSDATE,
JOB_TITLE VARCHAR2(50),
SALARY NUMBER(8,2),
COMMISSION_PCT NUMBER(3,2),
DEPT_ID NUMBER,
MANAGER_ID NUMBER,
CONSTRAINT FK_EMP_DEPT FOREIGN KEY (DEPT_ID)
REFERENCES DEPARTMENT(DEPT_ID)
);

-- STEP 4: Add self-referencing foreign key for manager
ALTER TABLE EMPLOYEE
ADD CONSTRAINT FK_EMP_MANAGER
FOREIGN KEY (MANAGER_ID) REFERENCES EMPLOYEE(EMP_ID);

-- STEP 5: Create PROJECT table
CREATE TABLE PROJECT (
PROJ_ID NUMBER PRIMARY KEY,
PROJ_NAME VARCHAR2(50) NOT NULL,
START_DATE DATE,
END_DATE DATE,
```

```
BUDGET NUMBER(10,2),
DEPT_ID NUMBER,
CONSTRAINT FK_PROJ_DEPT FOREIGN KEY (DEPT_ID)
REFERENCES DEPARTMENT(DEPT_ID)
);

-- STEP 6: Create EMP_PROJECT association table
CREATE TABLE EMP_PROJECT (
EMP_ID NUMBER,
PROJ_ID NUMBER,
ROLE VARCHAR2(30),
ALLOCATED_HOURS NUMBER(5),
CONSTRAINT PK_EMP_PROJECT PRIMARY KEY (EMP_ID, PROJ_ID),
CONSTRAINT FK_EP_EMP FOREIGN KEY (EMP_ID)
REFERENCES EMPLOYEE(EMP_ID),
CONSTRAINT FK_EP_PROJ FOREIGN KEY (PROJ_ID)
REFERENCES PROJECT(PROJ_ID)
);
```

1.1.3 Sample Data (Will Be Used in All Examples)

Now populate the tables:

```
-- Insert Departments
INSERT INTO DEPARTMENT (DEPT_ID, DEPT_NAME, LOCATION) VALUES (10, 'Computer Science',
     'Gandhinagar');
INSERT INTO DEPARTMENT (DEPT_ID, DEPT_NAME, LOCATION) VALUES (20, 'Cyber Security', '
    Gandhinagar');
INSERT INTO DEPARTMENT (DEPT_ID, DEPT_NAME, LOCATION) VALUES (30, 'Digital Forensics
    ', 'Delhi');

-- Insert Employees (managers first)
INSERT INTO EMPLOYEE (EMP_ID, FIRST_NAME, LAST_NAME, EMAIL, HIRE_DATE, JOB_TITLE,
SALARY, DEPT_ID)
VALUES (100, 'Anita', 'Shah', 'ANITA.SHAH@nfsu.ac.in',
DATE '2020-07-01', 'Head of Department', 120000, 10);

INSERT INTO EMPLOYEE (EMP_ID, FIRST_NAME, LAST_NAME, EMAIL, HIRE_DATE, JOB_TITLE,
SALARY, DEPT_ID)
VALUES (200, 'Bhavesh', 'Mehta', 'BHAVESH.MEHTA@nfsu.ac.in',
DATE '2021-01-15', 'Head of Department', 115000, 20);

-- Non-manager employees
INSERT INTO EMPLOYEE (EMP_ID, FIRST_NAME, LAST_NAME, EMAIL, PHONE_NO, HIRE_DATE,
JOB_TITLE, SALARY, COMMISSION_PCT, DEPT_ID, MANAGER_ID)
VALUES (101, 'Chetan', 'Patel', 'CHETAN.PATEL@nfsu.ac.in', '9876543210',
DATE '2022-06-10', 'Assistant Professor', 80000, NULL, 10, 100);

INSERT INTO EMPLOYEE (EMP_ID, FIRST_NAME, LAST_NAME, EMAIL, PHONE_NO, HIRE_DATE,
JOB_TITLE, SALARY, COMMISSION_PCT, DEPT_ID, MANAGER_ID)
VALUES (102, 'Deepa', 'Rao', 'DEEPA.RAO@nfsu.ac.in', '9876500001',
DATE '2023-02-01', 'Assistant Professor', 78000, 0.10, 10, 100);

INSERT INTO EMPLOYEE (EMP_ID, FIRST_NAME, LAST_NAME, EMAIL, PHONE_NO, HIRE_DATE,
```

```
JOB_TITLE, SALARY, COMMISSION_PCT, DEPT_ID, MANAGER_ID)
VALUES (201, 'Eshan', 'Kumar', 'ESHAN.KUMAR@nfsu.ac.in', '9876500002',
DATE '2022-09-20', 'Research Fellow', 65000, NULL, 20, 200);

INSERT INTO EMPLOYEE (EMP_ID, FIRST_NAME, LAST_NAME, EMAIL, PHONE_NO, HIRE_DATE,
JOB_TITLE, SALARY, COMMISSION_PCT, DEPT_ID, MANAGER_ID)
VALUES (202, 'Fatima', 'Syed', 'FATIMA.SYED@nfsu.ac.in', '9876500003',
DATE '2024-01-05', 'Teaching Assistant', 45000, NULL, 20, 200);

-- Insert Projects
INSERT INTO PROJECT (PROJ_ID, PROJ_NAME, START_DATE, END_DATE, BUDGET, DEPT_ID)
VALUES (1001, 'Web App Security Audit', DATE '2023-07-01', NULL, 500000, 20);

INSERT INTO PROJECT (PROJ_ID, PROJ_NAME, START_DATE, END_DATE, BUDGET, DEPT_ID)
VALUES (1002, 'Malware Analysis Lab Setup', DATE '2023-01-15', DATE '2023-12-31',
    800000, 30);

INSERT INTO PROJECT (PROJ_ID, PROJ_NAME, START_DATE, END_DATE, BUDGET, DEPT_ID)
VALUES (1003, 'Student Portal Upgrade', DATE '2024-06-01', NULL, 300000, 10);

-- Map employees to projects
INSERT INTO EMP_PROJECT (EMP_ID, PROJ_ID, ROLE, ALLOCATED_HOURS)
VALUES (101, 1003, 'Developer', 120);

INSERT INTO EMP_PROJECT (EMP_ID, PROJ_ID, ROLE, ALLOCATED_HOURS)
VALUES (102, 1003, 'Tester', 80);

INSERT INTO EMP_PROJECT (EMP_ID, PROJ_ID, ROLE, ALLOCATED_HOURS)
VALUES (201, 1001, 'Security Analyst', 150);

INSERT INTO EMP_PROJECT (EMP_ID, PROJ_ID, ROLE, ALLOCATED_HOURS)
VALUES (202, 1001, 'Report Writer', 60);

COMMIT;
```

You can verify the relations with:

```
SELECT * FROM DEPARTMENT;
SELECT * FROM EMPLOYEE;
SELECT * FROM PROJECT;
SELECT * FROM EMP_PROJECT;
```

——— End of the Document ———

1.2 DDL: Data Definition Language

1.2.1 What is DDL?

Data Definition Language (DDL) consists of SQL statements that define or change the structure of database objects:

- `CREATE` – create tables, indexes, views, etc.

- `ALTER` – change the structure of existing objects.
- `DROP` – remove objects from the database.
- `TRUNCATE` – remove all rows from a table (faster than DELETE, no WHERE).

DDL statements are:

- **auto-committed** in Oracle (once executed, changes cannot be rolled back),
- usually used by DBAs, but developers must also understand them.

1.2.2 Examples on Our Schema

Example 1.1 (Add a NOT NULL column with default) *Suppose we want to track whether an employee is active.*

```
ALTER TABLE EMPLOYEE
ADD (IS_ACTIVE CHAR(1) DEFAULT 'Y' NOT NULL);
```

Example 1.2 (Change column size) *If we expect longer project names:*

```
ALTER TABLE PROJECT
MODIFY (PROJ_NAME VARCHAR2(80));
```

Example 1.3 (Drop a constraint)

```
ALTER TABLE EMPLOYEE
DROP CONSTRAINT FK_EMP_MANAGER;
```

1.3 DML: Data Manipulation Language

1.3.1 What is DML?

Data Manipulation Language (DML) deals with the *rows* stored in tables:

- `INSERT` – add new rows,
- `UPDATE` – modify existing rows,
- `DELETE` – remove rows.

DML statements:

- are **transactional**: their effect is visible only after `COMMIT`,
- can be undone using `ROLLBACK` before commit.

1.3.2 INSERT Examples

```
-- Insert a new project for Cyber Security department
INSERT INTO PROJECT (PROJ_ID, PROJ_NAME, START_DATE, BUDGET, DEPT_ID)
VALUES (1004, 'Threat Intelligence Dashboard', DATE '2024-09-01', 600000, 20);
```

1.3.3 UPDATE Examples

```
-- Give a 10% salary hike to all employees in Computer Science
UPDATE EMPLOYEE
SET SALARY = SALARY * 1.10
WHERE DEPT_ID = 10;
```

1.3.4 DELETE Examples

```
-- Remove employees who have no department assigned (data cleanup)
DELETE FROM EMPLOYEE
WHERE DEPT_ID IS NULL;
```

1.4 Transaction Control: COMMIT and ROLLBACK

1.4.1 ACID Idea in Miniature

Even before Unit V, we informally see ACID in DML:

- **Atomicity**: either all changes in a transaction happen, or none.
- **Consistency**: database moves from one valid state to another.
- **Isolation**: concurrent transactions do not corrupt each other.
- **Durability**: committed changes survive failures.

1.4.2 Simple Transaction Scenario

```
-- Step 1: Start changes (no explicit BEGIN in SQL)
UPDATE EMPLOYEE
SET SALARY = SALARY + 5000
WHERE EMP_ID = 101;

UPDATE EMPLOYEE
SET SALARY = SALARY + 5000
WHERE EMP_ID = 102;

-- Step 2: Decide to undo
ROLLBACK;
```

After `ROLLBACK`, salaries of EMP_ID 101 and 102 return to original values.

1.4.3 Simple Transaction Timeline

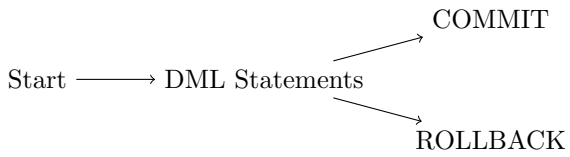


Figure 1.1: Transaction flow: DML followed by COMMIT or ROLLBACK

1.5 Retrieving Data Using SELECT

1.5.1 Basic SELECT Structure

`SELECT` ⟨column list⟩ `FROM` ⟨table⟩ `WHERE` ⟨condition⟩

Example 1.4 (All employees)

```
SELECT EMP_ID, FIRST_NAME, LAST_NAME, SALARY
FROM EMPLOYEE;
```

1.5.2 Restricting Data: WHERE Clause

```
-- Employees in Cyber Security (DEPT_ID = 20)
SELECT FIRST_NAME, LAST_NAME, JOB_TITLE, SALARY
FROM EMPLOYEE
WHERE DEPT_ID = 20;
```

```
-- Employees hired after 1st January 2023
SELECT FIRST_NAME, LAST_NAME, HIRE_DATE
FROM EMPLOYEE
WHERE HIRE_DATE > DATE '2023-01-01';
```

1.5.3 Sorting Data: ORDER BY

```
-- Highest salary first
SELECT FIRST_NAME, LAST_NAME, SALARY
FROM EMPLOYEE
ORDER BY SALARY DESC;
```

1.6 Single Row Functions

Single row functions operate on **one row at a time** and return **one value per row**. We will see:

- **Character functions** – Operate on text values (UPPER, LOWER, INITCAP, SUBSTR, INSTR, LPAD, RPAD, TRIM, REPLACE).
- **Date functions** – Perform operations on DATE values (MONTHS_BETWEEN, ADD_MONTHS, NEXT_DAY, LAST_DAY, ROUND, TRUNC).
- **Conversion functions** – Convert data types (TO_CHAR, TO_NUMBER, TO_DATE).
- **General functions** – Handle NULL values, multiple-value selection, and conditional logic:
  - **NVL(expr1, expr2)** – Returns `expr2` if `expr1` is NULL.
  - **NVL2(expr1, expr2, expr3)** – If `expr1` is NOT NULL, returns `expr2`; else returns `expr3`.
  - **COALESCE(expr1, expr2, ...)** – Returns the first non-NULL expression from the list.
  - **NULLIF(expr1, expr2)** – Returns NULL if both expressions are equal; otherwise returns `expr1`.
  - **CASE ... WHEN ... THEN ... ELSE ... END** – Conditional logic similar to IF-ELSE.

1.6.1 Character Functions

```
-- Display full name in upper case
SELECT UPPER(FIRST_NAME || ' ' || LAST_NAME) AS FULL_NAME_UPPER
FROM EMPLOYEE;
```

```
-- Extract first 3 characters of department name
SELECT DEPT_NAME,
SUBSTR(DEPT_NAME, 1, 3) AS SHORT_CODE
FROM DEPARTMENT;
```

```
-- Pad project names for formatted output
SELECT RPAD(PROJ_NAME, 30, '.') AS PADDED_NAME, BUDGET
FROM PROJECT;
```

1.6.2 Date Functions

```
-- How many months each employee has worked
SELECT FIRST_NAME, LAST_NAME,
MONTHS_BETWEEN(SYSDATE, HIRE_DATE) AS MONTHS_WORKED
FROM EMPLOYEE;
```

```
-- Next Monday from today
SELECT NEXT_DAY(SYSDATE, 'MONDAY') AS NEXT_MONDAY
FROM DUAL;
```

1.6.3 Conversion Functions

```
-- Format salary as currency-like string
SELECT FIRST_NAME,
TO_CHAR(SALARY, '999,999.00') AS SALARY_STR
FROM EMPLOYEE;
```

```
-- Convert string to date
SELECT TO_DATE('15-08-2025', 'DD-MM-YYYY') AS INDEPENDENCE_DAY
FROM DUAL;
```

1.6.4 General Functions

```
-- 1. NVL: Replace NULL commission with 0
SELECT FIRST_NAME, COMMISSION_PCT,
NVL(COMMISSION_PCT, 0) AS COMMISSION_VALUE
FROM EMPLOYEE;
```

```
-- 2. NVL2: Show different messages for NULL and NOT NULL cases
SELECT FIRST_NAME, COMMISSION_PCT,
NVL2(COMMISSION_PCT,
'Eligible for commission',
'No commission')
AS COMMISSION_STATUS
FROM EMPLOYEE;
```

```
-- 3. COALESCE: Return first NON-NULL value from the list
-- Priority: Commission -> Salary*0.05 -> 0
SELECT FIRST_NAME, COMMISSION_PCT, SALARY,
COALESCE(COMMISSION_PCT,
SALARY * 0.05,
0) AS FINAL_PAYOUT
FROM EMPLOYEE;
```

```
-- 4. NULLIF: Avoid division-by-zero
-- If commission_pct = 0 then NULLIF returns NULL -> Safe division
SELECT FIRST_NAME, COMMISSION_PCT,
100 / NULLIF(COMMISSION_PCT, 0) AS SAFE_DIV
FROM EMPLOYEE;
```

```
-- 5. CASE Expression: Salary band classification
SELECT FIRST_NAME, LAST_NAME, SALARY,
CASE
WHEN SALARY >= 100000 THEN 'High Salary'
WHEN SALARY BETWEEN 60000 AND 99999 THEN 'Medium Salary'
ELSE 'Low Salary'
END AS SALARY_BAND
FROM EMPLOYEE;
```

```
-- 6. CASE inside ORDER BY (Advanced)
-- Sort employees: High salaries first, then medium, then low
SELECT FIRST_NAME, SALARY
FROM EMPLOYEE
ORDER BY
CASE
WHEN SALARY >= 100000 THEN 3
WHEN SALARY BETWEEN 60000 AND 99999 THEN 2
ELSE 1
END DESC;
```

1.7 Connection to Upcoming Units

In the next units we will:

- Use the same tables for **group functions**, **GROUP BY**, **HAVING** and various types of **joins** (Unit IV).
- Build **subqueries** and **WITH** clause on top of this schema (Unit IV).
- Implement **transactions**, **PL/SQL blocks**, **cursors**, **triggers** and **packages** using these tables (Unit V).

——— End of the Document ———

"In God we trust. All others must bring data."

– W. Edwards Deming

.